SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0604 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO2: Neural Network Representation, Perceptron Model, Multilayer Perceptrons, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)

Assessment Tool Weightage: 20%

QUESTIONS: 15 **Total Marks:25**

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Sanjay S/192521058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sanjay S

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

1. Conceptual Understanding

A Perceptron is a basic neural network model used for binary classification. It learns a decision boundary to separate two classes using weights and a bias.

For a linearly separable dataset, the perceptron learning algorithm should converge in a finite number of updates if the standard update rule is implemented correctly.

The output is calculated as:

$$y = \begin{cases} 1, & w \cdot x + b \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

The weight update rule is:

$$w_{\text{new}} = w_{\text{old}} + \eta(y_{\text{true}} - y_{\text{pred}})x$$

$$b_{\text{new}} = b_{\text{old}} + \eta(y_{\text{true}} - y_{\text{pred}})$$

Here, x is the input, w is the weight vector, b is the bias, and η is the learning rate.

2. Accuracy of Answer: Possible Causes of Non-Convergence

1. Improper learning rate

- A very large learning rate can cause unstable, excessively large updates.
- A very small learning rate can make learning slow and appear not to converge within the allotted iterations.
- Solution: Start with a moderate learning rate, such as 0.01 or 0.1, and test different values.

2. Incorrect weight initialization

- Poorly initialized weights may delay learning.
- Incorrect dimensions or non-finite initial values can cause implementation errors.
- Solution: Initialize weights to small values or zeros and initialize the bias to zero.

3. Bias handling errors

- Omitting the bias may prevent the decision boundary from moving away from the origin.
- Forgetting to update the bias can cause classification errors.
- Solution: Update the bias whenever a misclassification occurs.

4. Flawed update rule

- Using the wrong sign in the update equation can move the decision boundary in the wrong direction.

- Updating weights even when predictions are correct can cause unnecessary changes.
- Incorrect target encoding, such as mixing labels 0/1 and −1/+1, can also cause errors.
- Solution: Verify the prediction, label encoding, and update equations carefully.

5. Data and implementation problems

- The dataset may not actually be linearly separable after preprocessing.
- Incorrect feature scaling, mismatched array dimensions, or wrong label indexing may prevent convergence.
- Solution: Validate the dataset, dimensions, labels, and preprocessing steps.

3. Application of Knowledge: Debugging Strategies

The following steps can help identify the root cause.

1. Verify linear separability: Plot the data for two features or use a trusted solver to check whether a separating hyperplane exists.
2. Check labels: Confirm that all target labels use the expected encoding.
3. Inspect updates: Print the weights, bias, prediction, and number of misclassified samples after each epoch.
4. Test a small dataset: Use a simple linearly separable dataset, such as the AND logic gate.
5. Verify the update rule: Confirm that weights and bias change only when a sample is misclassified.
6. Check stopping conditions: Stop when an entire epoch has zero misclassifications, or report a maximum-epoch warning if errors remain.
7. Check numerical values: Ensure that weights and inputs contain no NaN or infinite values.

4. Completeness of Response: Optimization Techniques

Technique	**Benefit**
Feature normalization	Helps prevent features with large numerical ranges from dominating.
Appropriate learning rate	Improves the stability and speed of updates.
Shuffling training samples	Reduces dependence on a fixed sample order.
Vectorized computation	Improves implementation efficiency.
Averaged perceptron	Can improve practical prediction performance by averaging weights across updates.
Pocket algorithm	Retains the best-performing weights when convergence is not achieved, especially for nonseparable data.

Important: The standard perceptron has a finite-convergence guarantee for linearly separable data under the correct update rule. Techniques such as the pocket algorithm are useful when the data are noisy or nonseparable, but they do not replace debugging a faulty implementation.

5. Clarity & Presentation: Conclusion

A perceptron may fail to converge despite a linearly separable dataset because of an incorrect learning rate, faulty weight or bias updates, inconsistent labels, or implementation errors. Debugging should begin by checking data separability, label encoding, and the mathematical update rule. With correct implementation, suitable initialization, feature scaling, and clear stopping conditions, the perceptron should converge reliably on linearly separable data.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

1. Conceptual Understanding

A Multilayer Perceptron (MLP) is a feedforward neural network containing an input layer, hidden layers, and an output layer. It learns by using the backpropagation algorithm to calculate errors and update weights to minimize the loss function.

Slow convergence occurs when the network takes too many iterations to learn. A model may also become trapped in a poor local minimum or a flat region of the loss surface, preventing further improvement.

2. Accuracy of Answer: Possible Causes

1. Vanishing gradients

- Gradients become extremely small as they propagate through multiple layers.
- Earlier layers learn very slowly.
- Solution: Use ReLU or suitable modern activation functions and consider residual connections for deep networks.

2. Inappropriate activation functions

- Sigmoid and tanh can saturate, producing very small gradients.
- Poor activation choices can slow learning.
- Solution: Use ReLU or Leaky ReLU in hidden layers and an appropriate output activation, such as sigmoid for binary classification.

3. Poor weight initialization

- Very large weights can cause unstable activations and gradients.
- Very small weights can reduce gradient magnitudes.
- Solution: Use He initialization for ReLU-based networks or Xavier initialization for tanh-based networks.

4. Suboptimal learning rate

- A very low learning rate causes slow progress.
- A very high learning rate may cause oscillations or overshooting.
- Solution: Tune the learning rate and use a suitable learning-rate schedule or adaptive optimizer.

5. Poorly scaled data

- Features with very different numerical ranges can make optimization inefficient.
- Solution: Standardize or normalize input features using statistics calculated from the training data.

3. Application of Knowledge: Step-by-Step Debugging

1. Inspect the dataset: Check missing values, incorrect labels, duplicate records, and class imbalance.
2. Normalize the inputs: Standardize numerical features to improve optimization.
3. Check activation functions: Verify that hidden-layer activations are appropriate and not frequently saturated.
4. Monitor gradients: Record gradient magnitudes for each layer. Extremely small gradients may indicate vanishing gradients.
5. Inspect weight initialization: Use Xavier or He initialization as appropriate.
6. Plot training and validation loss: Determine whether training is slow, unstable, or overfitting.
7. Tune the learning rate: Test several values and compare convergence speed and stability.
8. Verify backpropagation: Check the loss calculation, gradient equations, and weight-update implementation. Gradient checking with finite differences can identify coding errors.

4. Completeness of Response: Optimization Techniques

Technique	**Purpose**
Adaptive learning rates (Adam, RMSProp)	Adjust update sizes based on gradient information.
Momentum	Smooths updates and may accelerate progress in consistent directions.
Batch normalization	Stabilizes activation distributions and can improve training speed.
He/Xavier initialization	Helps maintain suitable activation and gradient magnitudes.
ReLU or Leaky ReLU	Reduces saturation problems in hidden layers.
Learning-rate scheduling	Reduces the learning rate as training progresses.
Early stopping	Stops training when validation performance no longer improves.

Recommended approach: Start with standardized input data, He initialization, ReLU hidden layers, Adam optimization, and a suitable learning rate. Monitor both training and validation loss, then adjust settings based on the observed problem.

5. Clarity & Presentation: Conclusion

Slow convergence in an MLP may result from vanishing gradients, unsuitable activation functions, poor weight initialization, inappropriate learning rates, or incorrectly scaled data. Systematic debugging helps identify the actual cause. Techniques such as Adam, momentum, normalization, and suitable initialization can improve training speed and stability.

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

1. Conceptual Understanding

A Genetic Algorithm (GA) is an optimization technique inspired by natural evolution. It searches a hypothesis space by maintaining a population of candidate solutions called chromosomes and improving them over generations.

The main operations are:

1. Selection: Selects candidate solutions based on fitness.
2. Crossover: Combines parent chromosomes to produce offspring.
3. Mutation: Randomly changes genes to explore new solutions.
4. Replacement: Forms the next generation from parents and offspring.

If a GA produces inconsistent or poor solutions across runs, it may have problems with chromosome encoding, fitness calculation, population diversity, or selection strategy.

2. Accuracy of Answer: Possible Causes

1. Improper chromosome encoding

- Chromosomes may not correctly represent valid solutions.
- Incorrect gene positions or data types can generate invalid hypotheses.
- Solution: Choose suitable binary, integer, or real-valued encoding and verify that every chromosome represents a valid candidate.

2. Incorrect fitness function

- The fitness function may reward poor solutions or measure the objective incorrectly.
- Constraints may be ignored, or the function may have the wrong optimization direction.
- Solution: Test the fitness function using known examples and verify whether higher fitness truly means a better solution.

3. Premature convergence

- The population becomes similar too early and stops exploring better solutions.

- Strong selection pressure can cause the best individuals to dominate quickly.
- Solution: Reduce selection pressure and introduce suitable mutation or diversity-preservation techniques.

4. Lack of population diversity

- Repeated chromosomes limit the search to a small part of the hypothesis space.
- The algorithm may repeatedly produce similar offspring.
- Solution: Increase population diversity through mutation, random immigrants, or diversity-aware selection.

5. Incorrect genetic operators

- Crossover may break valid chromosomes, while excessive or insufficient mutation may harm the search.
- Solution: Select operators appropriate to the encoding and preserve chromosome validity.

3. Application of Knowledge: Systematic Debugging

1. Verify chromosome encoding: Check gene positions, chromosome length, and allowed gene values.
2. Test the fitness function: Calculate fitness manually for a few known chromosomes and compare the results with the program.
3. Inspect each generation: Record the best, average, and worst fitness values.
4. Measure diversity: Count unique chromosomes or calculate a diversity score to detect premature convergence.
5. Check selection pressure: Examine whether a few individuals dominate the population too quickly.
6. Monitor mutation and crossover: Verify that the operators produce valid offspring and that their probabilities are applied correctly.
7. Repeat independent runs: Use different random seeds and compare best fitness, average fitness, and convergence speed.
8. Check stopping conditions: Ensure the algorithm does not stop too early or continue unnecessarily after progress has stalled.

4. Completeness of Response: Optimization Strategies

Technique	**Purpose**
Mutation-rate tuning	Maintains diversity and explores new solutions.
Selection-pressure adjustment	Prevents a few chromosomes from dominating too early.
Elitism	Preserves the best solutions in the next generation.
Larger population	Increases the range of candidate solutions explored.

Suitable crossover	Combines useful characteristics of parent solutions.
Fitness scaling	Prevents excessive dominance by individuals with unusually high fitness.
Diversity preservation	Reduces premature convergence.
Multiple independent runs	Helps evaluate the consistency and reliability of the search.

Recommended approach: Begin with correct chromosome encoding and fitness calculation. Then monitor population diversity and fitness trends, tune mutation and selection pressure, and use moderate elitism to preserve good solutions without eliminating exploration.

5. Clarity & Presentation: Conclusion

A genetic algorithm may produce inconsistent and suboptimal solutions because of incorrect chromosome encoding, a flawed fitness function, premature convergence, or insufficient population diversity. Systematic debugging helps identify the root cause. By tuning mutation, controlling selection pressure, preserving elite solutions, and testing multiple independent runs, the algorithm can achieve more reliable solutions and improve convergence speed.

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, overparameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

1. Conceptual Understanding

A Deep Neural Network (DNN) learns complex patterns from data using multiple hidden layers, while Genetic Programming (GP) evolves computer programs or mathematical expressions to find useful solutions.

Combining DNN and GP can improve prediction performance, but the system may suffer from:

- Overfitting: The model performs well on training data but poorly on unseen data.
- High computational cost: Training deep networks and evaluating many GP candidates requires significant time and memory.
- Overparameterization: Too many parameters increase model complexity and the risk of memorizing training examples.
- Insufficient training data: Limited examples prevent the model from learning patterns that generalize well.

The objective is to improve prediction accuracy while reducing computational requirements and overfitting.

2. Accuracy of Answer: Causes and Debugging

1. Excessive model complexity

- Too many hidden layers, neurons, or GP program branches can make the system unnecessarily complex.
- Solution: Reduce network size, limit GP tree depth, and remove unnecessary components.

2. Overparameterization

- A large number of trainable parameters can allow the model to memorize training examples.
- Solution: Use weight regularization, parameter sharing, and pruning.

3. Insufficient training data

- Small or unrepresentative datasets may not cover the range of real-world situations.
- Solution: Collect more representative data or apply suitable data augmentation.

4. Excessive GP search cost

- Evaluating a large population of complex GP programs across many generations consumes substantial computation.
- Solution: Limit program size, cache repeated evaluations, and use efficient fitness evaluation.

5. Poor training configuration

- Inappropriate learning rates, excessive training epochs, or incorrect validation procedures can worsen generalization.
- Solution: Tune hyperparameters, monitor validation performance, and use early stopping.

3. Application of Knowledge: Step-by-Step Debugging

1. Check data quality: Identify missing values, duplicates, incorrect labels, and class imbalance.
2. Split the dataset: Create training, validation, and test sets. Keep the test set separate until final evaluation.
3. Compare training and validation performance: High training accuracy with low validation accuracy indicates likely overfitting.
4. Inspect model complexity: Count DNN parameters and measure GP program depth, size, and evaluation time.
5. Check data sufficiency: Determine whether the training examples represent the expected real-world conditions.
6. Profile computation: Measure training time, memory consumption, and the time spent evaluating GP candidates.
7. Test a simpler baseline: Compare against a smaller neural network or a simpler prediction model.
8. Run controlled experiments: Apply one optimization technique at a time and compare validation performance and computational cost.

4. Completeness of Response: Optimization Techniques

Technique	**Purpose**
L1/L2 regularization	Penalizes unnecessary or excessively large weights and reduces overfitting.
Dropout	Randomly disables selected neurons during training to reduce co-adaptation.

Early stopping	Stops training when validation performance stops improving.
Pruning	Removes unnecessary weights, neurons, or GP program branches to reduce complexity.
Data augmentation	Creates suitable variations of training examples to improve generalization.
GP tree-depth limits	Prevents unnecessarily large and expensive programs.
Fitness caching	Avoids recalculating fitness for identical GP candidates.
Efficient evaluation models	Uses smaller models or inexpensive proxy evaluations to screen candidates before full evaluation.
Hyperparameter tuning	Finds suitable learning rates, model sizes, and regularization strengths.

Recommended approach: Begin with data-quality checks and training-versus-validation comparisons. Then use L2 regularization, dropout, early stopping, and a smaller network. For genetic programming, limit program depth, cache repeated fitness calculations, and evaluate promising candidates more thoroughly than poor candidates.

5. Clarity & Presentation: Conclusion

A deep neural network combined with genetic programming can overfit and become computationally expensive because of excessive model complexity, too many parameters, insufficient training data, and costly candidate evaluation. Debugging should focus on validation performance, model size, data quality, and computation profiling. Regularization, dropout, pruning, early stopping, and efficient GP evaluation can reduce overfitting and computational cost while improving generalization. The final model should be selected by balancing prediction accuracy, generalization, training time, and memory usage.

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

1. Conceptual Understanding

A Decision Tree is a supervised machine learning algorithm used for classification and regression. It makes predictions by splitting data into branches based on feature values and produces an output at a leaf node.

When a decision tree achieves high accuracy on training data but performs poorly on unseen data, it is likely suffering from overfitting. This means the model has learned noise or specific details of the training data instead of general patterns.

2. Accuracy of Answer: Possible Causes

1. Overfitting

- The tree memorizes training examples rather than learning general patterns.
- Solution: Apply pruning and restrict tree complexity.

2. Excessively deep tree structure

- A tree with too many levels can create rules specific to individual training samples.

- Solution: Limit the maximum depth and increase the minimum number of samples required for a split or leaf.

3. Noisy or incorrect data

- Incorrect labels, outliers, and measurement errors can lead to misleading splits.
- Solution: Clean the dataset, verify labels, and handle outliers appropriately.

4. Irrelevant features

- Unnecessary features may introduce misleading splits and increase model complexity.
- Solution: Apply feature selection and remove redundant attributes where appropriate.

5. Unrepresentative training data

- The training set may not represent the real-world population or may differ from the test distribution.
- Solution: Use representative, properly stratified data splits and investigate distribution differences.

3. Application of Knowledge: Debugging Steps

1. Compare training and test performance: A large gap between training and test accuracy suggests overfitting.
2. Inspect tree depth: Check the number of levels, leaves, and decision rules.
3. Check data quality: Identify missing values, incorrect labels, duplicates, and noisy observations.
4. Analyze feature importance: Identify irrelevant or redundant features and verify whether splits are meaningful.
5. Use cross-validation: Evaluate the model across multiple folds to estimate its generalization performance.
6. Inspect the confusion matrix: Identify which classes are frequently misclassified.
7. Check data leakage: Ensure that training features do not contain information derived from the target or test data.
8. Compare simpler models: Train a shallower tree and compare its validation performance with the original tree.

4. Completeness of Response: Optimization Methods

Technique	**Purpose**
Pre-pruning	Stops unnecessary tree growth by limiting depth or split conditions.
Post-pruning	Removes branches that do not improve validation performance.
Cross-validation	Helps select reliable hyperparameters and estimate generalization.
Feature selection	Removes irrelevant or redundant features.

Minimum leaf samples	Prevents leaves from representing too few training examples.
Class balancing	Helps when class imbalance affects performance.
Noise reduction	Improves the quality of training data.

Recommended approach: First compare training and validation performance. Then use cross-validation to tune maximum depth, minimum samples per leaf, and pruning strength. Select the model that performs consistently on validation data rather than choosing the tree with the highest training accuracy.

5. Clarity & Presentation: Conclusion

A decision tree with high training accuracy but poor unseen-data performance is likely overfitting because of excessive depth, noisy data, irrelevant features, or an unrepresentative training set. Debugging should include performance comparison, tree inspection, data-quality checks, and cross-validation. Pruning, feature selection, and complexity control can improve generalization and produce a more reliable decision tree.

Do you like this personality?